

Programming for Essential Digital Skills, Chapter 5

Christoph Walsh

2026

Assigning Multiple Values to a Variable

- Python has a number of ways you can assign multiple values to a single variable:
 - ▶ Lists
 - ▶ Tuples
 - ▶ Dictionaries
 - ▶ Sets (and Frozen Sets).

Each of these have different properties, which we will learn about.

Lists

Lists can contain any sequence of values, which could be integers, floats, strings, Booleans, or any combination of them. They can even contain lists as elements.

```
a = [2, 4, 6]
a = [1, 1.1, True, 'hello']
a = [1, 2, [3, 4]]
```

Arithmetic Operations on Lists

+ combines lists and * repeats lists (like with strings):

```
a = [2, 4, 6]  
b = [8, 10, 12]  
a + b
```

```
[2, 4, 6, 8, 10, 12]
```

```
3 * a
```

```
[2, 4, 6, 2, 4, 6, 2, 4, 6]
```

- and / do not work on lists.

Indexing

- We can extract elements of a list with indexing, which uses the order the elements appear.
- In Python (and many other languages like C), indexing starts at 0.
 - ▶ This means the 1st element has index 0, the 2nd element has index 1, and so on.

```
a = [2, 4, 6]  
a[0]
```

2

```
a[1]
```

4

```
a[2]
```

6

Mutability of Lists

We can change the value of an element its index:

```
a = [2, 4, 6]  
a[0] = 8  
a
```

[8, 4, 6]

Because lists have this property we say they are *mutable*.

List Slicing

To get a range of elements in a list we can use *slicing*:

```
a = [1, 2, 3, 4, 5]  
a[1:3]
```

```
[2, 3]
```

```
a[:2]
```

```
[1, 2]
```

```
a[2:]
```

```
[3, 4, 5]
```

```
a[:]
```

```
[1, 2, 3, 4, 5]
```

List Methods

Lists have *methods*, which are like functions but instead of providing the list as an argument to the function, we apply the method directly to the object.

We can add an element at the end of a list using the `append( )` method:

```
a = [2, 4, 6]  
a.append(8)  
a
```

```
[2, 4, 6, 8]
```

Notice how the method changes a *in place*.

List Methods

Removing an element (by index):

```
a = [2, 4, 6]  
a.pop(1)
```

4

a

[2, 6]

Sorting a list:

```
a = [6, 2, 4]  
a.sort()  
a
```

[2, 4, 6]

Iterating over Items in a List

- A useful thing we can do with lists is iterate over each element and perform an operation on those elements.
- For example, if we want to square each number:

```
a = [2, 4, 6]
for i in a:
    print(i ** 2)
```

```
4
16
36
```

Indentation Matters!

The following two code snippets only differ by indentation, but produce different output:

Snippet 1:

```
for i in a:
    print(i ** 2)
    print('hello')
```

Snippet 2:

```
for i in a:
    print(i ** 2)
print('hello')
```

`print('hello')` is part of the loop in case 1, and after the loop in case 2.

For Loops

If we want to save each squared number we could do:

```
a = [2, 4, 6]
b = []
for i in a:
    b.append(i ** 2)
b
```

[4, 16, 36]

List Comprehensions

However, an easier way to do this is to use *list comprehensions*:

```
a = [2, 4, 6]  
b = [i ** 2 for i in a]  
b
```

```
[4, 16, 36]
```

List Membership

We can check if a value is contained somewhere in a list using the `in` operator:

```
a = [2, 4, 6]  
4 in a
```

True

```
5 in a
```

False

Copying Lists

Consider the following:

```
a = [2, 4, 6]  
b = a  
a[0] = 8  
b
```

[8, 4, 6]

A change in a automatically changes b!

Copying Lists

To avoid this we can do:

```
a = [2, 4, 6]  
b = a[:]  
a[0] = 8  
b
```

[2, 4, 6]

Two Types of Copying

- *Deep copy*: This copies a to b and recursively copies all of its elements, resulting in a completely independent object.
- *Shallow copy*: This copies a to b but does not recursively copy its elements. Instead, it only copies the references to the elements in a (like the address for where in the computer's memory those elements are stored). This means that changes to elements of a will affect the elements of b.

Tuples

- A tuple is like a list with the important difference that they are *immutable*. Once we create them we cannot change them.
- We can create tuples with parentheses instead of square brackets:

```
a = (2, 4, 6)
```

- Indexing, iterating and other operations for lists also work with tuples.
- However, appending, popping, or altering values is not possible.

Tuple Assignment

Suppose we wanted to do the following:

```
x = ['a', 'b', 'c']  
x_0 = x[0]  
x_1 = x[1]  
x_2 = x[2]
```

We can shorten this code by using *tuple assignment*:

```
(x_0, x_1, x_2) = x
```

Dictionaries

- A dictionary is a collection that maps *keys* to *values*.
- The keys can be strings or integers, and the values can be any type.

```
suppliers = {100001 : 'ABC Ltd.', 100002 : 'EFG Ltd.'}
```

Extracting Values from a Dictionary

- Dictionaries are unordered, so we cannot extract elements with indexing, e.g. with `suppliers[0]`.
- Instead we extract elements using the keys:

```
suppliers[100001]
```

```
'ABC Ltd.'
```

Adding or Modifying Values to a Dictionary

We can add keys and values to a dictionary:

```
suppliers[100003] = 'HIJ Ltd.'
```

We can also modify values:

```
suppliers[100003] = 'KLM Ltd.'
```

Getting all Keys or Values

We can get all the keys or values with:

```
suppliers.keys()
```

```
dict_keys([100001, 100002, 100003])
```

```
suppliers.values()
```

```
dict_values(['ABC Ltd.', 'EFG Ltd.', 'KLM Ltd.'])
```

These can be converted to a list. For example:

```
list(suppliers.keys())
```

```
[100001, 100002, 100003]
```

Iterating Over Keys

Iterating over a dictionary implicitly iterates over the keys:

```
for key in suppliers:  
    print('Supplier with ID ' + str(key) + ' is ' + suppliers[key])
```

Supplier with ID 100001 is ABC Ltd.

Supplier with ID 100002 is EFG Ltd.

Supplier with ID 100003 is KLM Ltd.

Sets

- A set is another way to store multiple items into a single variable.
- Sets are unordered and unindexed. This means you cannot extract individual elements using their index like a list, nor by their key like a dictionary.
- We create sets using {}:

```
myset = {"apple", "banana", "cherry"}  
myset
```

```
{'apple', 'cherry', 'banana'}
```

Sets

Duplicates are not possible:

```
myset = {"apple", "banana", "cherry", "cherry"}  
myset
```

```
{'apple', 'cherry', 'banana'}
```

But adding elements is:

```
myset = {"apple", "banana", "cherry"}  
myset.add("pear")  
myset
```

```
{'apple', 'cherry', 'banana', 'pear'}
```

We can remove with `myset.remove("apple")`.

Sets

We can use sets to get all the unique elements in a list:

```
fruits = ["apple", "apple", "apple", "banana", "cherry", "banana"]  
set(fruits)
```

```
{'apple', 'cherry', 'banana'}
```

We can also iterate over sets just like with lists and tuples.

Set Operations

We can also do mathematical set operations like $\cap$ (intersection), $\cup$ (union) and set differences:

```
set_a = {1, 2, 4, 6, 8, 9}  
set_b = {2, 3, 5, 7, 8}  
set_a.intersection(set_b)
```

{8, 2}

```
set_a.union(set_b)
```

{1, 2, 3, 4, 5, 6, 7, 8, 9}

```
set_a.difference(set_b)
```

{1, 4, 6, 9}

Frozen Sets

You want to make an immutable set, you can use the `frozenset()` command:

```
myset = frozenset([1, 2, 3])
```

Looking back at the objects we've seen, we can summarize:

- list: mutable and ordered.
- tuple: immutable and ordered.
- set: mutable and unordered.
- frozenset: immutable and unordered.